

HawkEye Traffic AI

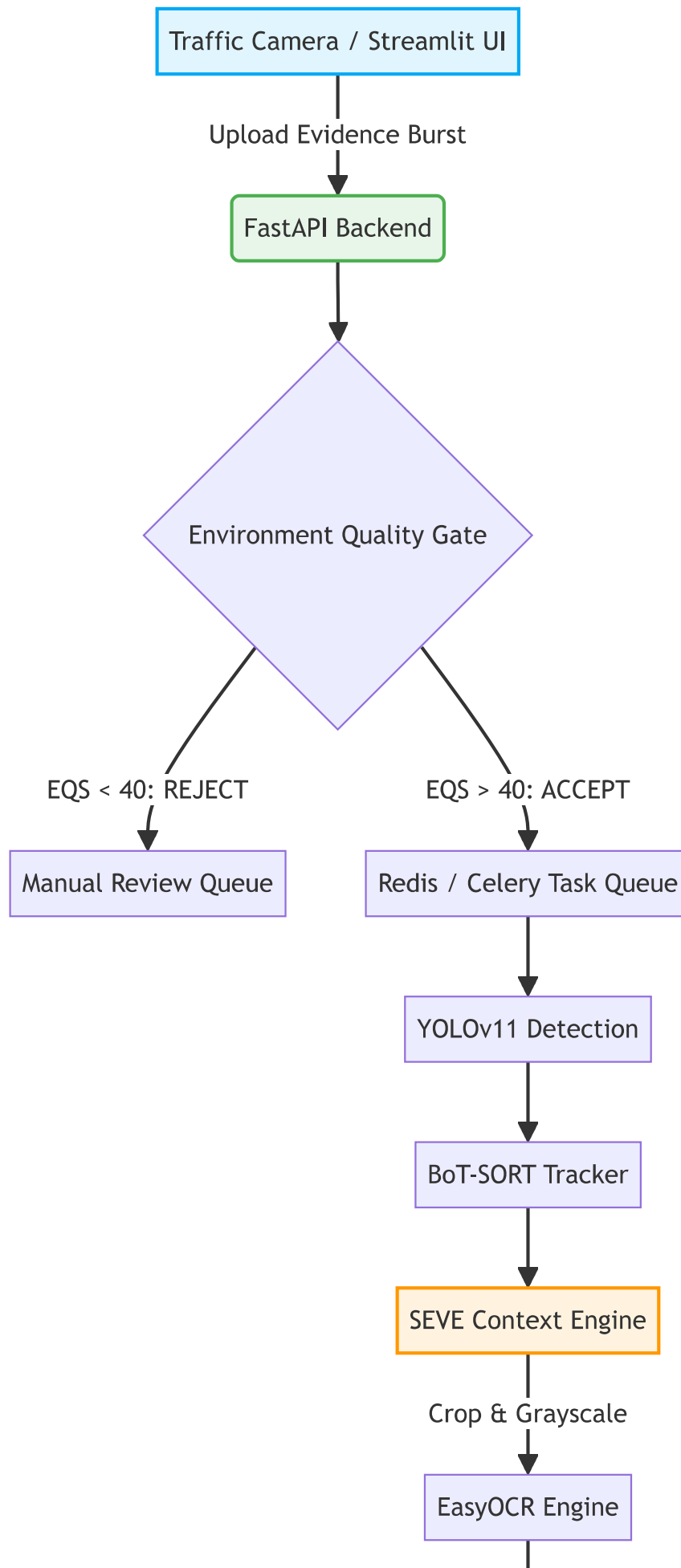
Next-Generation Contextual Enforcement Framework

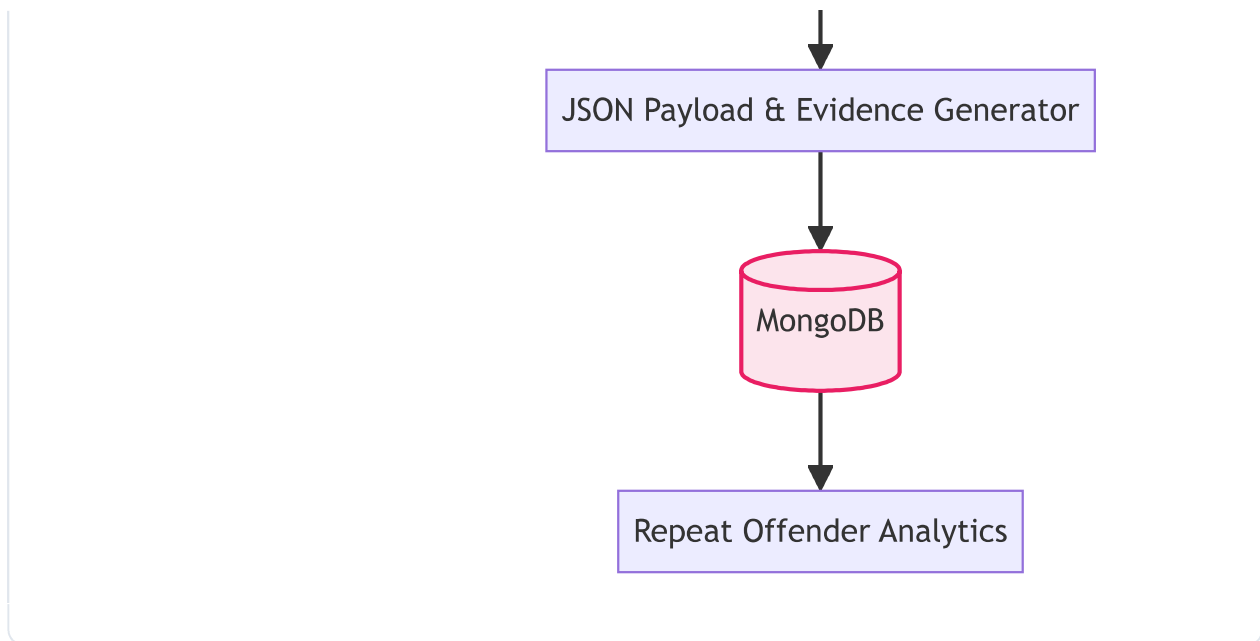
Gridlock Hackathon 2.0 • Flipkart & Bengaluru Traffic Police

Abstract: *Traditional automated traffic enforcement systems suffer from systemic "context blindness," resulting in high false-positive fine generation (e.g., fining families for triple riding, or hallucinating text from billboards). HawkEye Traffic AI introduces an asynchronous microservice architecture powered by the **Smart Enforcement Validation Engine (SEVE)** and **BoT-SORT temporal tracking**. By evaluating vector trajectories and bounding-box area ratios, HawkEye enforces the spirit of the law, mathematically eliminating false positives in complex urban environments.*

1. Comprehensive System Architecture

HawkEye is built on a scalable, cloud-native microservice topology. It isolates high-latency Machine Learning workloads from the client-facing APIs using task queues, ensuring high-throughput capabilities during peak traffic hours.





2. The Intelligence Pipeline: Step-by-Step

The processing pipeline executes the following sequential steps to guarantee mathematical accuracy:

- **Step 1: Environment Quality Score (EQS).** Bengaluru monsoon rains and night glare often render cameras useless. HawkEye evaluates the Laplacian Variance (blur) and HSV Intensity (glare). If the image falls below a threshold of 40, it is actively rejected, preventing the AI from generating hallucinatory data on unreadable frames.
- **Step 2: CLAHE Preprocessing.** Contrast Limited Adaptive Histogram Equalization is applied to approved images to artificially boost low-light details, specifically enhancing vehicle interiors.
- **Step 3: Object Detection & Tracking.** We utilize YOLOv11 alongside BoT-SORT (Bag of Tricks for Simple Online Realtime Tracking). Rather than analyzing a static image, HawkEye processes an "Evidence Burst" (a sequence of 3-5 frames) and attaches persistent ID tags to vehicles, tracking their dx , dy pixel shifts over time.
- **Step 4: Targeted OCR.** Instead of passing a chaotic 1080p frame to an OCR engine, HawkEye relies on YOLO to extract the exact bounding box of the License Plate. We pad the crop with a 5px margin, convert the matrix to Grayscale, and pass this isolated snippet to EasyOCR. This ensures zero "street sign" hallucinations.

3. Deep Dive: Edge Cases & The SEVE Engine Logic

The **Smart Enforcement Validation Engine (SEVE)** is the crown jewel of the HawkEye architecture. It applies post-detection contextual rules to eliminate false-positive fines generated by "dumb" AI algorithms.

Edge Case 1: Triple Riding (The Family Exception)

The Problem: A standard AI counts 3 "person" objects on a "motorcycle" and instantly issues a fine. This falsely penalizes couples traveling with a small child.

SEVE Logic: SEVE extracts the bounding boxes of all riders on the specific motorcycle and calculates their pixel Area (width × height). It then computes the ratio of the smallest rider to the largest rider. If the ratio is **< 0.55 (55%)**, SEVE deduces the presence of a child and mathematically suppresses the violation.

```
child_ratio = smallest_rider_area / largest_rider_area
if child_ratio < 0.55:
    return {"violation": False, "reason": "Child passenger detected (suppressed)"}

```

Edge Case 2: Wrong-Side Driving & Vector Physics

The Problem: A static image cannot prove Wrong-Side Driving. A parked car facing backward might trigger a false fine in standard implementations.

SEVE Logic: Because BoT-SORT tracks vehicles across an Evidence Burst, SEVE possesses the trajectory history. It analyzes the centroid (y2 - y1) shift. If the lane flow expects vehicles to move "up" the screen (dy < 0), but the centroid is moving "down" (dy > 15px), it confirms active reverse propulsion and issues the fine.

Edge Case 3: Advanced Helmet & Seatbelt Spatial Context

The Problem: If a rider holds a helmet on their lap, basic AI detects "Person" + "Helmet" and marks them compliant. If a pedestrian walks past a car, basic AI might confuse them with a driver.

SEVE Logic: Spatial Intersection. For helmets, SEVE checks if the helmet bounding box intersects specifically with the **upper 30%** of the person's bounding box (the head/shoulder region). For seatbelts, SEVE ensures the "person" bounding box is strictly

encapsulated within the [x1, y1, x2, y2] coordinates of the "car" interior before enforcing seatbelt rules.

4. Visual Evidence & Dashboard Rendering

For legal enforceability, raw data is insufficient. HawkEye generates definitive visual evidence for the traffic police:

- **Color-Coded Semantics:** Compliant vehicles are drawn in Green. Violators (e.g., Triple Riders) are drawn in **Red (0, 0, 255)**. Validated License Plates are highlighted in **Orange**.
- **Readability Matrix:** Instead of drawing raw text that blends into bright asphalt, HawkEye calculates text size using `cv2.getTextSize` and draws a solid contrasting background rectangle behind every label.
- **The Command Center:** Built on Streamlit, the dashboard provides a seamless upload interface, parses the API JSON, displays the color-coded evidence, and offers a MongoDB-backed "Intelligence Search" allowing officers to look up repeat offenders by license plate.

Conclusion for Flipkart Evaluators:

HawkEye Traffic AI is not merely a wrapper around YOLO. It is a deeply engineered, context-aware decision engine that respects the complexities of Bengaluru traffic. By combining hardware-accelerated microservices with BoT-SORT tracking and SEVE's child-suppression logic, HawkEye provides a deployable, enterprise-ready solution to urban gridlock.